

Documentación del software `fibonacci_photonics` 2.0

Emanuel Orozco Gallego

28 de septiembre de 2026

Resumen

Manual independiente de la biblioteca Python que reproduce Reyes-Gómez *et al.*, Phys. Rev. B **81**, 153101 (2010), con la matriz de transferencia (TMM) y la expansión en ondas planas (PWE).

Índice

1. Arquitectura	2
2. Instalación	2
3. Uso	2
4. API principal	2
5. Modelo físico	3
6. TMM	3
7. Fibonacci	3
8. Dispersión	3
9. Pruebas	3
10.Reproducción de figuras	3
11.Solución de problemas	4
12.Extensiones	4

1. Arquitectura

El paquete vive en `src/fibonacci_photonics/`.

Subpaquete	Responsabilidad
physics	materiales, TE/TM, S_m , F_m , L_m , SuperlatticeSpec
solvers	contrato DispersionSolver, TMMSolver, PWESolver
analysis	bordes de banda, subbandas plasmónicas, anchos vs θ
config	esquemas Pydantic de los YAML
benchmark	comparación y tiempos TMM vs PWE
reproduction, studies	Figs. 1–6 y estudios numéricos
io, viz	resultados, metadatos, tablas LaTeX, estilo de figuras

Los parámetros científicos no están en los scripts: están en `configs/figure_0N.yaml` y se validan al cargarlos. El diseño completo está en *Arquitectura, flujo y ejecución del proyecto*.

2. Instalación

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt # = pip install -e ".[dev]"
```

Python ≥ 3.10 . Dependencias: NumPy, SciPy, Matplotlib, PyYAML y Pydantic 2; el extra dev añade pytest, Ruff, mypy y types-PyYAML.

3. Uso

```
make tests lint
python scripts/run_figure.py figure_01 --method tmm # una figura
python scripts/run_figure.py all --method both # todas, ambos metodos
fibonacci-figure 5 --method pwe --workers 8 # comando instalado
python scripts/comparison/run_all.py # TMM vs PWE
make documents verify
```

Para un orden nuevo de Fibonacci, cambie `fibonacci_orders` en el YAML. Para TM, ponga `polarization: TM`. Para otro Drude, edite `omega_e_over_2pi_ghz` y `omega_m_over_2pi_ghz`.

4. API principal

```
import numpy as np
from fibonacci_photonics import PWESolver, TMMSolver, get_solver, load_spec

spec = load_spec() # superred del articulo
nu = np.linspace(0.5, 5.0, 2001) # GHz, > 0 y creciente
scan = TMMSolver(spec).scan(4, np.pi / 6, nu, "TE")
pwe = get_solver("pwe", spec, harmonics_per_layer=16)
r = pwe.semitrace(4, np.pi / 6, nu, "TE")
report = pwe.convergence_report(4, np.pi / 6, nu[:200])
```

- `solver.semitrace`, `solver.scan` y `solver.evaluator` tienen la misma firma en los dos métodos y devuelven los mismos objetos.

- `TMMSolver(spec, algorithm="recurrence"|"product")`: las dos vías de R_m deben coincidir.
- `PWESolver(spec, harmonics_per_layer, rule="inverse"|"laurent", workers)`: $n_{\text{máx}} = hF_m$; `convergence_report` compara con 1.5 veces más ondas planas y emite `ConvergenceWarning` si no converge.
- `close_band_edges(solver, scan)`: bordes $|R| = 1$ y puntas refinados.
- `detect_plasmon_modes(scan, nu_min_ghz, nu_max_ghz, bandwidth_versus_angle, compare_scans)`.
- `sequence(m), paper_fibonacci(m), build_cell(m, a, b)`.
- `load_spec(path), load_figure(figure_id)`: YAML validados.

Las entradas inválidas (frecuencias ≤ 0 o no finitas, mallas no crecientes, $\theta \notin [0, \pi/2]$, órdenes negativos, armónicos < 1) lanzan `InvalidParameterError`; los YAML inválidos, `ConfigError`.

5. Modelo físico

Aire $\varepsilon = \mu = 1$. Drude $\varepsilon = 1 - \omega_e^2/\omega^2$, $\mu = 1 - \omega_m^2/\omega^2$. Frecuencia graficada: $\nu = \omega/2\pi$ en GHz.

6. TMM

Matriz de capa unimodular con $\chi = \mu$ (TE) o ε (TM). Producto $T = M_N \cdots M_1$. Recurrencia $R_m = 2R_{m-1}R_{m-2} - R_{m-3}$. $q = (\omega/c)n_A \sin \theta$ (el paper omite ω/c).

7. Fibonacci

$$S_0 = B, S_1 = A, S_m = S_{m-1}S_{m-2}, F_0 = F_1 = 1, N_B = F_{m-2}, L_m = F_{m-1}a + F_{m-2}b.$$

8. Dispersión

Banda si $|R| \leq 1 + 10^{-12}$. $kL_m/\pi = \arccos(R)/\pi$. Los puntos con $|R| > 1$ se dejan como NaN, no se recortan en silencio.

9. Pruebas

- `tests/unit/`: contrato común, validaciones, esquemas Pydantic, Fourier, matrices de transferencia, PWE frente a TMM, bordes y análisis de bandas.
- `tests/analytic/`: límite homogéneo, bicapa de cuarto de onda ($R = -5/3$), $\det T = 1$, invariante de Kohmoto, TE=TM a $\theta = 0$, dualidad $\varepsilon \leftrightarrow \mu$, Parseval, razón áurea.
- `tests/regression/`: valores de referencia previos a la refactorización y $N_{\text{modos}} = F_{m-2}$.

10. Reproducción de figuras

Cada `results/tmm/figure_ON/` contiene `data/`, `output/` y `README.md`. Las versiones PWE están en `results/pwe/figure_ON/` y la comparación en `results/comparison/`. Las imágenes finales son PDF, PNG (300 dpi) y, para la TMM, SVG.

11. Solución de problemas

- `InvalidParameterError` con $\nu = 0$: el Drude diverge; no incluya el origen en el barrido.
- $\mu_B = 0$ o $\varepsilon_B = 0$: polo; el escáner enmascara una vecindad relativa 10^{-6} .
- $|R|$ siempre ≤ 1 a $\theta = 0$ si $\omega_e = \omega_m$: no es un bug, es adaptación de impedancia.
- `ConvergenceWarning`: aumente `harmonics_per_layer` o use la TMM cerca de ν_m .
- Fig. 6: el original grafica $\nu(\theta)$ relleno, no $\Delta\nu$; el caption del PRB está incompleto (parámetros inferidos de las Figs. 3–5).
- `ImportError`: instale el paquete (`pip install -e .`) o use `PYTHONPATH=src`.

12. Extensiones

Nuevo orden: `YAML`. Nueva polarización: `Polarization.TM`. Nuevo material: implemente `epsilon_mu(omega)` con la misma interfaz que `DrudeMetamaterial`. Nuevo método numérico: implemente `semitrace`, `scan` y `evaluator` (protocolo `DispersionSolver`) y todo el post-proceso funciona sin cambios.